Module 3 — Laboratory: Your First Interactive Web Page

Vistula University · IT Fundamentals · Laboratory assignment

Estimated time: 60–90 minutes **Mode:** Work in pairs (you already have a partner from Module 2)

Prerequisites

Before you start, you need:

- A GitHub account and a project repository from Module 2 (any repo with an `index.html` is fine).
- GitHub Pages already enabled for your repo (Settings → Pages → branch: `main`).
- A web browser (Chrome, Firefox, or Edge) and a text editor (VS Code recommended).
- Basic git commands from Module 2: `git switch -c`, `git add`, `git commit`, `git push`.

If something does not work, ask your teacher or your partner — asking is a skill, not a weakness.

Goal

You will create `tasks.html` — a small interactive **Student Task Manager**. The page will have a form, a task list, and it will remember your tasks even after you refresh the page.

Task 1 — Create a feature branch (5 min)

Never work directly on `main`. In your terminal, inside your repository folder:

```
git switch main
git pull
git switch -c feature/tasks-page
```

You are now on a new branch called `feature/tasks-page`. All work in this lab goes here.

Task 2 — Create the HTML page with a form (15 min)

Create a new file `tasks.html` in the root of your repository. It must contain:

1. A **title input field** (`<input type="text">`) — the name of the task.
2. A **due-date field** (`<input type="date">`) — when the task must be done.
3. A **priority select** (`<select>` with options: Low, Medium, High).
4. An **"Add" button**.
5. An **empty list or table** (`<ul id="taskList">` or `<table id="taskTable">`) where tasks will appear.
6. A place for **error messages**, for example `<p id="error" class="error"></p>`.

Start like this and finish the form yourself:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Student Task Manager</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>My Tasks</h1>
  <!-- your form goes here -->
  <!-- your task list goes here -->
  <script src="tasks.js"></script>
</body>
</html>
```

Add a few lines of CSS in `style.css` so the page does not look like 1995. A colored heading and some spacing are enough.

Task 3 — Add the JavaScript (20 min)

Create `tasks.js`. It must do three things:

a) Read the form and add the task to the list. When the user clicks "Add" (or submits the form):

```
const title = document.getElementById("title").value;
const dueDate = document.getElementById("dueDate").value;
const priority = document.getElementById("priority").value;
```

Then create a new list item (or table row) with this data and add it to the list. Show all three fields: title, due date, priority.

b) Validate the input before adding.

- If the **title is empty** (or only spaces), show a visible error message, for example "Please enter a task title." in the error paragraph, and do NOT add the task.
- If the title is **longer than 60 characters**, show "Title is too long (max 60 characters)." and do NOT add the task.
- Clear the error message when a valid task is added.

Hint: `title.trim() === ""` checks for empty input; `title.trim().length > 60` checks the length.

c) Save tasks in localStorage.

localStorage is a small storage space that the browser gives to your page. It survives a page refresh — and that is exactly what we want. After every change, save the whole list:

```
localStorage.setItem("tasks", JSON.stringify(tasks));
```

And when the page opens, load them back:

```
const tasks = JSON.parse(localStorage.getItem("tasks") || "[]");
```

(JSON comes from Module 3 theory: we turn the list into text to store it, and back into a list to use it.)

IMPORTANT — localStorage is NOT a shared database

Read this carefully, it is a common misunderstanding:

- **localStorage lives only in THIS browser, on THIS device.**
- Your tasks saved in Chrome on your laptop are **invisible** in Firefox, on your phone, or on your partner's computer.
- **No other user can see your tasks.** There is no server involved — the data never leaves your computer.
- This is why our page is a **static website**: all the "memory" is in one browser.

Real sharing — tasks visible to all users, on all devices — needs a **backend with a database**. We will build exactly that in **Module 4 using Supabase**. Today's localStorage is a first step, not the final destination.

Task 4 — Test your page (10 min)

Open `tasks.html` in your browser (double-click the file, or use VS Code "Live Server"). Check:

- [ ] Adding a task with all three fields works.
- [ ] An empty title shows an error and adds nothing.
- [ ] A title of 100 characters shows the length error.
- [ ] After pressing F5 (refresh), all your tasks are still in the list.
- [ ] After adding a task, the form fields are cleared.

Fix anything that fails before you commit.

Task 5 — Commit, pull request, review, merge (15 min)

Use the git workflow from Module 2:

```
git add tasks.html tasks.js style.css
git commit -m "Add interactive task manager page with localStorage"
git push -u origin feature/tasks-page
```

Then on GitHub:

1. Open a **Pull Request** from `feature/tasks-page` to `main`. Write one or two sentences describing what you built.
2. Ask your **partner to review** the code. The reviewer leaves at least one comment — a question or a suggestion. (A real review, not just "LGTM".)
3. Fix what the reviewer found, push again if needed.
4. **Merge** the pull request, then delete the feature branch.

Task 6 — Publish on GitHub Pages (5 min)

Because `main` already serves GitHub Pages from Module 2, your merged file is published automatically. Wait 1–2 minutes, then open:

```
https://YOUR-USERNAME.github.io/YOUR-REPO-NAME/tasks.html
```

Test the page once more **online**. Refresh it. The tasks are still there — because localStorage is tied to this address in this browser.

Expected Deliverables

By the end of the lab, your repository must contain:

- `tasks.html` — the page with the form (title, due date, priority), the Add button, and the task list.
- `tasks.js` — reading the form, validation (empty title, max length), rendering the list, saving/loading localStorage.
- `style.css` — updated with styles for the new page.
- One merged **Pull Request** with at least one partner review comment.

Submission Instructions

Submit through the course platform on the assignment page (Assignments section): paste your GitHub repository URL / deployed app URL / PR URL as required.

1. **Repository URL** — the GitHub link to your project.
2. **GitHub Pages URL** — the direct link to your live `tasks.html`.
3. **Names and student IDs** of both pair members.

Deadline: end of the lab session. Late submissions lose points according to the course rules.